

Outbreak: Stochastic Survival Lab

Comparación de un proceso de Poisson y un proceso de renovación generado mediante el método polar

Estudiante(s): _____

Curso de Modelación y Simulación

Docente: _____

20 de agosto de 2026



Figura 1: Concepto visual de la zona de contención. Imagen generada para este proyecto.

Resumen

Este trabajo desarrolla una simulación estocástica de supervivencia ante infectados. Las llegadas se modelan principalmente mediante un proceso de Poisson homogéneo, cuyos interarribos exponenciales se generan con transformada inversa. Como contraste se implementa un proceso de renovación con interarribos lognormales; las normales necesarias se obtienen con el método polar de Marsaglia. Ambos modelos se parametrizan con el mismo tiempo medio entre llegadas. La dinámica de combate se resuelve con paso temporal fijo y la probabilidad de supervivencia se estima por Monte Carlo con intervalos de Wilson al 95 %. La aplicación Streamlit integra controles, escena tridimensional procedural, trazas temporales y experimentos reproducibles.

Índice

1	Introducción	3
2	Planteamiento del problema	3
2.1	Pregunta de investigación	3
2.2	Objetivo general	3
2.3	Objetivos específicos	3
3	Fundamento matemático	3
3.1	Proceso de Poisson homogéneo	3
3.2	Método polar y alternativa lognormal	4
3.3	Acumulación de llegadas	4
4	Modelo de la simulación	5
4.1	Variables	5
4.2	Reglas operativas	5
4.3	Parámetros base	6
5	Diseño experimental	6
6	Resultados de referencia	6
7	Validación y verificación	7
8	Interfaz y tecnología 3D	7
9	Supuestos y limitaciones	8
10	Conclusiones	8
11	Trabajo futuro	8

1 Introducción

En un videojuego de supervivencia, crear un enemigo cada cantidad fija de segundos produce una secuencia predecible. Un modelo estocástico representa mejor la incertidumbre de una situación donde se conoce una frecuencia media, pero no el instante exacto de la siguiente aparición.

El proyecto estudia la relación entre esa incertidumbre y la supervivencia de un protagonista. Además de una realización individual, se ejecutan cientos de partidas para cuantificar variabilidad y evitar conclusiones basadas en un caso aislado.

2 Planteamiento del problema

2.1 Pregunta de investigación

¿Cómo cambian la congestión de enemigos y la probabilidad de supervivencia cuando varían la tasa y la distribución de los tiempos entre llegadas?

2.2 Objetivo general

Modelar las apariciones aleatorias de infectados e integrar el proceso de llegadas con reglas de movimiento y combate para estimar la probabilidad de que el protagonista sobreviva un horizonte temporal finito.

2.3 Objetivos específicos

1. Generar interarribos exponenciales mediante transformada inversa.
2. Implementar el método polar de Marsaglia y construir interarribos lognormales positivos.
3. Comparar ambos modelos bajo una tasa media común.
4. Estimar probabilidades mediante Monte Carlo e informar incertidumbre.
5. Visualizar el estado del sistema en una escena 3D interactiva.
6. Garantizar reproducibilidad mediante semillas y pruebas automatizadas.

3 Fundamento matemático

3.1 Proceso de Poisson homogéneo

Sea $N(t)$ el número de llegadas hasta el tiempo t . Para una tasa constante $\lambda > 0$:

$$N(t) \sim \text{Poisson}(\lambda t), \tag{1}$$

$$P\{N(t) = k\} = e^{-\lambda t} \frac{(\lambda t)^k}{k!}, \quad k = 0, 1, 2, \dots \quad (2)$$

Una propiedad diagn3stica es la igualdad entre media y varianza:

$$\mathbb{E}[N(t)] = \text{Var}[N(t)] = \lambda t. \quad (3)$$

Los interarribos Δ_i son independientes y siguen una distribuci3n Exponencial(λ):

$$f_{\Delta}(x) = \lambda e^{-\lambda x}, \quad x \geq 0. \quad (4)$$

Si $U_i \sim U(0, 1)$, la transformada inversa produce

$$\boxed{\Delta_i = -\frac{\ln(U_i)}{\lambda}}. \quad (5)$$

Este modelo presenta falta de memoria y coeficiente de variaci3n igual a uno; por ello puede producir agrupaciones de llegadas y pausas largas.

3.2 M3todo polar y alternativa lognormal

El m3todo polar es un algoritmo de generaci3n de normales, no un proceso de llegadas. Se sortean $V_1, V_2 \sim U(-1, 1)$ y se calcula $S = V_1^2 + V_2^2$. Los pares con $S \notin (0, 1)$ se rechazan. Para un par aceptado:

$$Z_1 = V_1 \sqrt{\frac{-2 \ln S}{S}}, \quad Z_2 = V_2 \sqrt{\frac{-2 \ln S}{S}}. \quad (6)$$

Ambos resultados siguen $N(0, 1)$. Como un tiempo no puede ser negativo, se usa la transformaci3n lognormal

$$\Delta_i = \exp(\mu + \sigma Z_i). \quad (7)$$

Sea c el coeficiente de variaci3n. Para conservar $\mathbb{E}[\Delta] = 1/\lambda$ se eligen

$$\sigma^2 = \ln(1 + c^2), \quad \mu = \ln(1/\lambda) - \frac{\sigma^2}{2}. \quad (8)$$

Con $c < 1$, las llegadas son m3s regulares que en Poisson. El conteo resultante es un proceso de renovaci3n y no sigue, en general, una distribuci3n Poisson.

3.3 Acumulaci3n de llegadas

Para ambos modelos se construyen los instantes

$$t_0 = 0, \quad t_i = t_{i-1} + \Delta_i. \quad (9)$$

Se conservan aquellos t_i dentro del horizonte de la misión. Si el protagonista cae antes del final, la telemetría de juego cuenta solamente las llegadas ocurridas antes de la derrota; el calendario completo se mantiene por separado para evitar la inconsistencia de la versión inicial.

4 Modelo de la simulación

4.1 Variables

Cuadro 1: Clasificación de variables del sistema.

Categoría	Variables
Entradas	λ , duración, modelo, CV, HP, DPS, alcance, velocidad, paso temporal y semilla.
Estado	HP del jugador, enemigos activos, posiciones, HP individual y tiempo.
Aleatorias	Interarribos, ángulo de aparición, radio, velocidad, HP y DPS de cada infectado.
Salidas	Supervivencia, tiempo resistido, llegadas, eliminados, remanentes y pico de concurrencia.

4.2 Reglas operativas

El protagonista permanece en el centro, ataca un objetivo a la vez y prioriza al enemigo vivo más cercano dentro de su alcance. Los infectados avanzan radialmente. Al alcanzar el radio de contacto, sus tasas de daño se suman. La misión concluye al llegar al horizonte T o cuando el HP llega a cero.

El tiempo se discretiza con $dt = 0.05$ s por defecto. Reducir dt disminuye el error temporal a costa de mayor cómputo. El motor conserva una lista de enemigos activos y evita recorrer, en cada paso, entidades que todavía no han aparecido.

4.3 Parámetros base

Cuadro 2: Escenario base reproducible.

Parámetro	Valor	Unidad
Horizonte	90	s
Tasa	39	llegadas/min
CV Polar	0.45	adimensional
HP protagonista	110	HP
DPS protagonista	42	HP/s
Alcance	10	m
HP infectado	42	HP
Velocidad infectado	1.55	m/s
DPS infectado	9	HP/s
Paso temporal	0.05	s
Semilla	22193	—

5 Diseño experimental

Para una configuración fija se ejecutan R partidas con semillas independientes. Si I_r vale uno cuando la partida r termina con supervivencia:

$$\hat{p} = \frac{1}{R} \sum_{r=1}^R I_r. \quad (10)$$

La incertidumbre se expresa con un intervalo de Wilson al 95 %, preferido a la aproximación normal simple cuando la proporción está cerca de cero o uno.

Para comparar modelos se utiliza una semilla maestra que produce el mismo vector de semillas de corrida para Poisson y Polar-lognormal. Además, dentro de cada partida se separa el flujo aleatorio de llegadas del flujo de atributos enemigos. Este diseño pareado reduce diferencias accidentales.

6 Resultados de referencia

Se ejecutaron 400 partidas por modelo con el escenario base. La Tabla 3 contiene resultados reproducibles con la versión entregada.

Cuadro 3: Comparación Monte Carlo del escenario base, 400 partidas por modelo.

Modelo	$\hat{p}$	IC 95 %	Tiempo medio (s)	Pico medio
Poisson	0.965	[0.942, 0.979]	88.526	9.483
Polar-lognormal	1.000	[0.990, 1.000]	90.000	6.483

Poisson produjo mayor concurrencia en este escenario, aunque ambos modelos comparten una tasa media de 39 llegadas/min. El resultado se explica por la mayor variabilidad exponencial: las rachas cortas elevan la presión simultánea. El modelo Polar-lognormal con $CV = 0.45$ distribuye las llegadas de forma más regular.

Estos resultados no significan que Polar sea siempre más seguro ni que Poisson sea “mejor” por producir determinada probabilidad. Poisson se considera el modelo principal por su interpretabilidad, parsimonia y correspondencia con los supuestos de llegadas independientes a tasa constante. Si los datos reales mostraran regularidad o dependencia, otro proceso podría ser más adecuado.

7 Validación y verificación

La entrega incluye nueve pruebas automáticas:

- reproducibilidad del generador y de la partida completa;
- cercanía de media y varianza Poisson a λT ;
- media lognormal cercana a $1/\lambda$ y positividad;
- separación entre llegadas programadas y ocurridas tras una derrota;
- rechazo de geometrías inválidas;
- igualdad de semillas en la comparación pareada;
- estabilidad cualitativa al reducir dt ;
- validez básica del intervalo de Wilson.

Las pruebas se ejecutan con:

```
python -m unittest discover -s tests -v
```

8 Interfaz y tecnología 3D

Streamlit controla formularios, estado y caché de experimentos. La simulación solo se ejecuta cuando el usuario pulsa un botón explícito. Plotly representa el terreno y mallas triangulares procedurales de personajes, edificios, vehículos y barricadas.

Pygame resulta apropiado para una aplicación de escritorio con superficies, entrada y bucle de juego; el 3D requeriría PyOpenGL y una ventana separada. Panda3D es una alternativa futura para un ejecutable 3D independiente. Se eligió Plotly porque mantiene visualización, controles y análisis en la misma aplicación web.

9 Supuestos y limitaciones

1. La tasa es constante dentro de una partida.
2. No existen colisiones entre infectados ni evitación de obstáculos.
3. Las ruinas son parte visual del escenario y no modifican trayectorias.
4. El protagonista es estacionario y enfoca un objetivo a la vez.
5. El combate es una aproximación discreta dependiente de dt .
6. El histograma de una sola misión puede sufrir censura por el horizonte.
7. Los resultados Monte Carlo dependen del escenario y del número de repeticiones; por eso se reportan intervalos de confianza.

10 Conclusiones

El proyecto conecta una variable aleatoria continua con un mecanismo visible del juego: cada interarribo determina cuánto falta para el siguiente infectado. El proceso de Poisson ofrece una formulación directa, interpretable y verificable. La comparación Polar-lognormal muestra que igualar solamente la tasa no iguala la congestión ni la supervivencia; la forma de la distribución también importa.

La versión final mejora la coherencia de telemetría, incorpora incertidumbre, separa fuentes aleatorias, valida parámetros, optimiza el recorrido de entidades y agrega pruebas. La escena 3D facilita la comunicación del modelo, mientras que el informe y la interfaz dejan explícitos los supuestos y límites.

11 Trabajo futuro

Se propone estimar una tasa no homogénea $\lambda(t)$ para oleadas, permitir movimiento del protagonista, incorporar colisiones y cobertura, calibrar parámetros con datos observados y desarrollar una versión independiente en Panda3D con modelos animados y sonido.

Referencias

- [1] S. M. Ross, *Simulation*, Academic Press.
- [2] L. Devroye, *Non-Uniform Random Variate Generation*, Springer.
- [3] Pygame Community, *Pygame Documentation*.
<https://www.pygame.org/docs/>
- [4] Panda3D, *Introduction to Panda3D*.
<https://docs.panda3d.org/1.10/python/introduction/index>

- [5] Plotly, *3D Mesh Plots in Python*.
<https://plotly.com/python/3d-mesh/>
- [6] Streamlit, *Custom Components*.
<https://docs.streamlit.io/develop/concepts/custom-components>